Summary

I used the ChatGPT 5.1 Thinking model under the study mode to have it go over LoRA before I start the homework assignment. It did a good job going over the contents within the lecture and asked questions to ensure my understanding. Under study mode, it continues to ask me questions that are directly related to what was taught in lecture without needing reminders or any prompting that I am studying before completing a homework assignment. I would say the model does a good job at proposing questions that solidify basic understanding but does not increase in difficulty to be like a homework or discussion problem. However, it is able to adapt questions based on how well the user understands the content.

I also gave the same starting responses to the ChatGPT 5.1 Thinking model without study mode. While it did give a few questions, it was less interactive and only listed a few questions in the end after going over the lecture content. Instead of asking for an answer to the questions, it states that if the user has any more questions, they can respond.

For studying the content, using the model under study mode provides a more interactive experience by continuously asking questions and adapting to how well the user understands the content. However, the regular model is suitable for covering all the content in the lecture.

ChatGPT 5.1 Thinking model with Study mode

Me: Explain a topic to me

ChatGPT:
Sure 🙂
What topic do you want to dig into? (Math, physics, history, programming, something else?)

Me: I attached a lecture note that goes over LoRA. I want to ensure I have a good understanding of it before I begin my homeworks. (with Lecture 22 notes attached)

ChatGPT:

It did a very good job finding exactly which parts of the lecture to look at and going over a brief overview of LoRA (equivalent to how much was in lecture)

LoRA in your notes is really "**low-rank weight updates instead of full fine-tuning**."
Let me walk through the picture in your lecture and translate it.

---

1. What problem is LoRA solving?

Full fine-tuning = you take a huge pretrained model and update **all** its weights (W_0).

- Pros: very flexible, great performance.

- Cons: millions–billions of parameters to store per task + expensive to train.

Your earlier lecture pages talked about *soft prompts / prefixes* as one parameter-efficient trick; LoRA is another one, but it changes the **weights** instead of just the input tokens.

---

2. Core idea in the slide with the big matrix (W_0)

Look at page 5 where there's a big block labeled (W_0) and little colored blocks (A) and (B).

references the notes a lot when i think the model should be able to draw this visualization themselves to be more clear

- (W_0) = original pretrained weight matrix
  (for example, in an attention layer or MLP, could be (d_\text{out} \times d_\text{in})).

- In normal fine-tuning you'd learn a full update (\Delta W) of the same size.

LoRA says:

> **Don't learn a full (\Delta W). Constrain it to be low-rank.**

Mathematically:

1. Keep (W_0) **frozen**.

2. Learn a small update (\Delta W) such that
   [
   W = W_0 + \Delta W
   ]

3. Parameterize (\Delta W) as
   [
   \Delta W = B A
   ]
   where:

   - (A \in \mathbb{R}^{r \times d_\text{in}})

   - (B \in \mathbb{R}^{d_\text{out} \times r})

- (r) is the **rank** (a small number, like 4, 8, 16, …).

Because (r \ll \min(d_\text{in}, d_\text{out})),
(\Delta W) is low-rank and you only store (A) and (B), not a full matrix.

Parameter count comparison

- Full matrix: (d_\text{out} \cdot d_\text{in}) parameters.
- LoRA update: (r(d_\text{in} + d_\text{out})) parameters.

For typical sizes, this is *much* smaller.

---

3. How it looks in the forward pass

Suppose an input vector (x) goes through this layer.

- Original model: (y = W_0 x)
- With LoRA:
  [
  y = (W_0 + B A) x = W_0 x + B(Ax)
  ]

Implementation trick (what happens in code / diagram):

- You don't actually construct (W_0 + BA).
- You compute (u = A x) (projects into an r-dim space),
- then (v = B u) (projects back),
- and add it: (y = W_0 x + v).

Backprop only updates **A and B**; (W_0) stays fixed.

Often they also scale the LoRA path by a factor (\alpha / r) so its initial effect is small but trainable.

---

4. Where do we insert LoRA?

On page 5 the notes mention using LoRA in **MLP and attention (Q, K, V, O)** weights.

Typical choices:

- Pick a subset of matrices (e.g., only (W_Q) and (W_V)).
- For each chosen matrix, replace it with "(W_0) + LoRA(A,B)".

This gives you:

- A **shared base model** (W_0) for all tasks.
- Tiny **adapters** (the LoRA A/B for each task) that you can store and swap in/out.

---

5. What happens *after* training?

Once LoRA training is done (bottom of that LoRA slide):

- Option 1: **Keep it factorized** as (W_0) plus LoRA(A,B) — good when you want to ship many small adapters.
- Option 2: **Merge** the update into the weights:
  (W_\text{new} = W_0 + B A) and then throw away A,B.

The notes explicitly say you can "merge (W_0) and (\Delta W)" after LoRA finetuning.

---

6. Initialization & learning rate (page 6)

That next page talks about:

- Initializing LoRA so that initially (BA \approx 0), meaning the model starts off behaving exactly like the pretrained one.

- Tuning the **learning rate** for LoRA separately from the base model's pretraining LR; typical choices in the notes for Adam are around (10^{-4})–(5 \times 10^{-4}) or so (order-of-magnitude guideline).

So: start with "no change" and let training slowly adjust the low-rank part.

---

7. Big picture intuition

Conceptually:

- LoRA assumes that "the right tweak" to a big pretrained layer lives in a **low-dimensional subspace**.

- Instead of moving in **all** directions in parameter space (full (\Delta W)), we only move in a skinny r-dimensional subspace, controlled by A and B.

- This makes training cheaper, uses way fewer task-specific parameters, and lets you store lots of task adapters for one big base model.

immediately asks questions about LoRA instead of allowing the user to ask questions about the lecture content. personally, i would have wanted the model to ask if i had any more questions so i could tell it to explain LoRA more in-depth and go over things not covered in lecture.

Your turn (single quick check)

Say a layer has a (4096 \times 4096) weight matrix, and you use LoRA with rank (r = 8).
Without calculating exact numbers, **which has more parameters**: the original weight matrix or the LoRA update (A and B), and *why*?

Me: The original weight matrix will have more parameters because it has 4096 * 4096 parameters total when the LoRA update only has 4096 * 8 * 2 parameters total

ChatGPT:

Yes, exactly right ✅

- Original: (4096 \times 4096 \approx 16{,}000{,}000) params

- LoRA update: (4096 \times 8 + 8 \times 4096 = 2 \times 4096 \times 8 \approx 65{,}000) params

So the LoRA part is *tiny* compared to the base weight.

To get you homework-ready, there are three core pieces you really want to be solid on:

1. **The math form of LoRA**
2. **How it changes the forward pass**
3. **What the rank (r) does (tradeoff between performance & parameter count)**

it does ask questions related to these core pieces but they were very straigtforward

We already touched (1), so let's firm up (2) with one quick step.

We start with a linear layer taking input (x \in \mathbb{R}^{d_{\text{in}}}):

[
y = (W_0 + BA)x = W_0 x + B(Ax)
]

Here:

- (W_0 \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}) is frozen
- (A \in \mathbb{R}^{r \times d_{\text{in}}})
- (B \in \mathbb{R}^{d_{\text{out}} \times r})

So you:

1. Project **down**: (u = A x) (into r-dim space)
2. Project **up**: (v = B u)
3. Add: (y = W_0 x + v)

Only (A) and (B) get gradients.

Tiny shape-check (this is very homework-relevant)

Suppose a LoRA-ified linear layer maps 512 ⟶ 1024, and you choose rank (r = 4).

- What are the shapes of:
    - (W_0)
    - (A)
    - (B)

Try to write them as “rows × columns” (e.g., (1024 \times 512)), and then I’ll check and we can talk about how this maps almost 1:1 to PyTorch code.

Me: W_0 = 512 x 1024
A = 512 x 4
B = 4 x 1024

ChatGPT:

Nice try — you’ve got the right *idea* but all three shapes are flipped 🙂

If the layer maps **512 ⟶ 1024**, that means:

- Input dim (d_{\text{in}} = 512)
- Output dim (d_{\text{out}} = 1024)
- Rank (r = 4)

For a standard linear layer (no LoRA), PyTorch uses weights shaped:

- (W_0:; d_{\text{out}} \times d_{\text{in}} = 1024 \times 512)

LoRA factors:

- (A:; r \times d_{\text{in}} = 4 \times 512)

- (B:; d_{\text{out}} \times r = 1024 \times 4)

You can sanity-check by multiplying:

- (A x): ((4 \times 512) \cdot (512) \to (4))
- (B(Ax)): ((1024 \times 4) \cdot (4) \to (1024))

So the LoRA path produces a vector in (\mathbb{R}^{1024}), same as (W_0 x), and you can add them.

Now let's nail one more key idea: **the role of the rank (r).**

High-level tradeoff:

- Smaller (r) ⟶ fewer parameters, more restriction, might underfit.
- Larger (r) ⟶ more expressive updates, more parameters, closer to full fine-tuning.

Here's a quick check for you:

If you increase (r) from 4 to 64 on that same 512 ⟶ 1024 layer, what happens to:

1. The number of LoRA parameters (qualitatively: smaller, same, or larger)?
2. The expressive power of (\Delta W = BA) (more constrained or less constrained)?

Answer in words, not formulas.

Me: 1. larger
2. less constrained

ChatGPT:

Perfect 💯 — that's exactly right:

1. **More parameters** when (r) goes up
2. **Less constrained** (more expressive) update (\Delta W)

That’s the core tradeoff.

At this point you already understand:

- The **math form**: (\Delta W = BA)
- The **forward pass**: (y = W_0 x + B(Ax))
- The **rank tradeoff**: small (r) = tiny, constrained updates; big (r) = richer, larger updates

Now I want to fill in just a few remaining “homework-critical” pieces:

1. Where LoRA is used in a **Transformer**
2. How **training** works (what’s frozen vs trainable)
3. Why we **initialize** it the way we do

I’ll walk through each quickly, then give you a tiny check question.

---

1. Where does LoRA plug into a Transformer?

Typically, we add LoRA to **linear layers in attention and/or MLP**:

- Attention: (W_Q, W_K, W_V, W_O)
- MLP: the big projection matrices (like (W_1, W_2))

For each chosen weight (W), we replace it with:

[
W_0 ;+; BA
]

Your lecture likely shows this at the Q/K/V matrices in self-attention (because they’re very important for behavior but not the largest matrices).

---

2. What's actually trained?

During LoRA fine-tuning:

- **Frozen**: All original weights (W_0) in the base model
- **Trainable**: Only the LoRA parameters (A, B) (and sometimes layer norms / biases depending on the setup)

So gradients only flow into the small low-rank pieces. That's why LoRA is:

- Parameter-efficient
- Cheap to store per task
- Easy to swap between tasks (just swap the A/B matrices)

---

3. Initialization & scaling

To not ruin the pretrained model at the start, LoRA does:

- Initialize (A) and (B) so that (BA \approx 0)
    - Often: (A) randomly initialized, (B) initialized to **zero**, so initially (\Delta W = 0)
- Sometimes multiply the LoRA path by a scaling factor (\alpha / r)
    - So effective update is (\Delta W = \frac{\alpha}{r} BA)
    - This keeps the LoRA contribution small and stable early in training

So at step 0, the model behaves like the original pretrained model; as training progresses, the low-rank path starts to "bend" the behavior.

Quick check (one question)

Imagine you *forgot* to initialize (B) to zero and instead made both (A) and (B) big random matrices.

What bad thing could happen to your pretrained model's behavior at the very start of training?

Answer in words (no math needed).

Me: im not too sure

ChatGPT:

Totally fine, this one's more "intuition" than formulas 😊

If both (A) and (B) start as big random matrices, then:

- (\Delta W = BA) is **not** close to 0
- So your effective weight is (W_0 + \Delta W), where (\Delta W) is a **big random perturbation**
- That means at step 0 the model's outputs are **heavily distorted**, almost like you took the pretrained model and added noisy weights everywhere

So instead of "starting from a good pretrained model and gently nudging it," you're kind of **wrecking the pretrained behavior immediately** and hoping training can fix it. That can:

- Hurt performance a lot at the beginning
- Make optimization less stable or slower

That's why we like to initialize so (\Delta W \approx 0): we keep the good pretrained behavior and only gradually adjust it.

Quick check, just yes/no in your own words:

it is able to adapt based on how comfortable i understand the content since i said i was not sure of the answer to the previous question

Does it make sense why we *want* (\Delta W) to be near 0 at initialization, so the model starts off behaving like the original pretrained one?

Me: yes

this response took a while to generate and it seems like it was not responsive after i said "yes". i believe these short answers are harder for the model to parse because they need to remember what the previous response given to me was and also calibrate more much of. the content i understand.

ChatGPT:

Awesome 🌟

Let's lock this in with a quick **teach-back**, because that's the best way to see if you're homework-ready.

> Imagine a classmate asks:
> **"What is LoRA and why is it parameter-efficient compared to full fine-tuning?"**
> Can you answer in **3–5 sentences**, in your own words?

Don't worry about being perfect — just give your best explanation, and then I'll help you tighten it up and fill any gaps.

Me:
LoRA allows for more efficient fine-tuning by creating two matrices for every layer and trains on the parameters of the two matrices instead of all the parameters in the layer. It is efficient because the number of parameters are in terms of the rank which is usually quite small. There is an A matrix that transforms the input dimension to some rank size and a B matrix that takes the rank size and outputs the output dimensions.

ChatGPT:

This is a really good explanation 👍

A couple tiny tweaks to make it "exam ready":

- It's not necessarily *every* layer—LoRA is added to specific weight matrices (like (W_Q, W_V), or MLP weights) while the original weight (W_0) is **frozen**.
- The key phrase your prof likes (see the LoRA page) is that the update (\Delta W) is **low-rank** and is written as the product of two small matrices (B A), where
    - (A:) input dim (\rightarrow) rank (r)
    - (B:) rank (r) (\rightarrow) output dim.

A polished version of your answer could be:

> LoRA is a parameter-efficient fine-tuning method where we freeze the original weight matrix (W_0) and only learn a low-rank update (\Delta W = BA). The matrices (A) and (B) map input → rank (r) and rank (r) → output, so the number of trainable parameters scales with the small rank (r) instead of the full layer size. This lets us adapt the model using far fewer parameters while keeping the pretrained weights intact.

💡 Quick check: can you write a single equation that shows how the *new* weight (W) relates to (W_0), (A), and (B)?

the questions are quite limited and they go in circles a lot. it seems like it is restricting itself to the lecture content and not expanding more or giving more detailed explanations.

ChatGPT 5.1 Thinking model without Study mode

Me: Explain a topic to me

response to very first prompt that was asked in study mode but regular mode answers directly and explained a random topic

ChatGPT:

Let's do **a quick, clean explain of "overfitting vs. underfitting" in machine learning** — super general and useful.

---

1. The basic idea

You train a model on data. Three things can happen:

- **Underfitting** – the model is too simple, it doesn't even do well on the training data.
- **Overfitting** – the model is too "memorize-y", it does great on training data but bad on new data.
- **Good generalization** – the sweet spot: does well on both training and new data.

Think of it like studying for an exam:

- **Underfitting**: You skimmed one page of the textbook → you miss obvious questions.
- **Overfitting**: You memorize the practice exam answers word-for-word → you ace that practice test but panic on a slightly different real exam.
- **Generalization**: You actually learned the concepts → you can handle new but similar questions.

---

2. How it looks with a curve

Imagine a scatter of points that roughly follow a curve.

- **Underfit model**: a straight line that clearly misses the trend.

- **Overfit model**: a crazy wiggly curve that perfectly passes through every point, including noise.
- **Good model**: a smooth curve that doesn't hit every point exactly but captures the main pattern.

Training error vs. test error:

- Underfitting:
    - **High training error**, **high test error**
- Overfitting:
    - **Low training error**, **high test error**
- Good generalization:
    - **Low training error**, **low test error**

---

3. Why overfitting happens

Overfitting happens when the model has **too much capacity** relative to:

- The amount of data
- The amount of noise in the data

Examples:

- Using a **giant neural net** on a tiny dataset
- Using a **high-degree polynomial** to fit just a few points

The model can "explain" all the little random quirks of the training set, but those quirks don't appear in new data.

---

4. Tools to fight overfitting (regularization)

Regularization = ways to force the model to be simpler / less extreme.

Common methods:

- **L2 regularization (weight decay)**
  Penalize large weights. Loss becomes
  [
  \text{Loss} = \text{Training loss} + \lambda \sum_i w_i^2
  ]
  This nudges weights to be smaller → smoother behavior.

- **L1 regularization**
  Uses (|w_i|) instead of (w_i^2) → encourages many weights to be exactly zero (sparsity).

- **Dropout**
  During training, randomly "turn off" some neurons each step. Prevents co-adapting and forces robustness.

- **Early stopping**
  Watch validation loss:

  - Train model

  - Stop once validation loss starts going up (even if training loss is still going down).
    This prevents the model from fully memorizing the training data.

- **More data / data augmentation**
  If the model is too strong for your dataset, giving it more examples (or augmented examples) makes it harder to overfit.

---

5. How you actually detect overfitting in practice

You split data into:

- **Training set**

- **Validation set**

- (Sometimes) **Test set**

Then you watch:

- Training loss goes ↓
- Validation loss goes:
    - ↓ then ↑ → you're starting to overfit
    - ↓ and stays low → yay, good generalization
    - Stays high → you're underfitting

If you'd like, next I can **walk through a tiny numeric example** (like fitting a polynomial of different degrees) *or* switch to a totally different topic (e.g. backprop, online algorithms, or something non-CS). Just tell me what you're in the mood for.

Me: I attached a lecture note that goes over LoRA. I want to ensure I have a good understanding of it before I begin my homeworks. (with Lecture 22 notes attached)

ChatGPT:

Here's a clean walkthrough of LoRA that lines up with your lecture notes.

similar to the overview given during study mode but i think it goes more in-depth into the content

---

1. What problem is LoRA solving?

Before LoRA, your options to adapt a big pretrained model were:

1. **Full fine-tuning –** update *all* weights →
    - Great quality
    - But huge memory + storage (you must store a full copy per task)

2. **Prompt / prefix tuning** – keep the model frozen, learn only a small number of extra input tokens / prefixes →
    - Very parameter-efficient
    - But sometimes not as strong, or tricky for some tasks

LoRA (Low-Rank Adaptation) is a middle ground:

> Keep the original weights **frozen**, and learn a **low-rank update** to certain weight matrices.

So you only store a small number of extra parameters per task, but you're *actually modifying internal layers*, not just the input.

---

2. What does LoRA do to a single weight matrix?

Take some weight matrix in the pretrained model, say $W_0 \in \mathbb{R}^{d \times k}$.
In normal fine-tuning you'd learn a full update ΔW of the same size.

LoRA says:

1. **Decompose the update as low-rank:**
   $$\Delta W = BA$$
   where
    - $A \in \mathbb{R}^{r \times k}$
    - $B \in \mathbb{R}^{d \times r}$
    - **rank $r \ll \min(d, k)$** (e.g. 4, 8, 16…)
2. **New effective weight:**
   $$W = W_0 + \Delta W = W_0 + BA$$

3. **Forward pass for an input vector (x):**
   [
   y = W x = W_0 x + B(Ax)
   ]
   Intuition:

   - (A) projects the features into a small **bottleneck** of dimension (r),
   - (B) projects back to the original dimension.

So instead of learning (d \times k) parameters, you learn (r \times k + d \times r) parameters, which is much smaller when (r) is small.

---

3. How many parameters are we saving?

- Full update: (d \times k) parameters.
- LoRA update: (r(k + d)) parameters.

Example: suppose (d = k = 4096), (r = 8):

- Full matrix: (4096 \times 4096 \approx 16.8)M params.
- LoRA: (8(4096 + 4096) = 8 \times 8192 = 65{,}536) params.

So you go from **millions** to **tens of thousands** per matrix.

In large transformers you apply LoRA only to specific matrices (e.g. the attention Q/K/V/O projections and maybe the MLPs), so the total extra parameters per task stay tiny compared to the base model.

---

4. Training details (important for homework)

For each LoRA-ified layer:

1. **Freeze (W_0).**
    - You *do not* update the original pretrained weights.
2. **Introduce A and B and only train those.**
    - Optimizer sees (A) and (B) as learnable parameters.
    - Everywhere else is `requires_grad=False`.
3. **Initialization:**
    - Common (and likely what your notes mention):
        - Initialize (A = 0)
        - Initialize (B) with a small random Gaussian
    - Why? Because then initially
      [
      \Delta W = B A = 0
      \Rightarrow
      W = W_0
      ]
      So at step 0 your model behaves exactly like the pretrained model – nice and stable.
4. **Scaling factor α (often in implementations):**
   Sometimes we scale the LoRA part:
   [
   y = W_0 x + \frac{\alpha}{r} B A x
   ]
    - (\alpha) is a hyperparameter (e.g. 8, 16, 32…)
    - (\alpha/r) controls the *strength* of the LoRA update.
5. **After training (merging):**

   If you want one compact model (e.g. for deployment), you can fold the update in:
   [
   W^* = W_0 + BA
   ]

and then just use (W^*) as the new weight matrix with no extra LoRA logic.

---

5. Where is LoRA applied in a transformer?

Typical choices (depends on the homework, but conceptually):

- **Attention layers:**
    - Query / Key / Value / Output projection matrices
- **Feedforward (MLP) layers:**
    - The big linear layers inside the MLP block

You don't normally LoRA everything; you choose a subset of matrices that give good performance–vs–cost trade-off.

---

6. How is this different from soft prompts / prefixes?

From earlier in your notes:

- **Soft prompts / prefix tuning**:
    - You freeze the whole model.
    - You add *learnable tokens or key/value prefixes* at the input or per layer.
    - The adaptation lives in those extra tokens, not in the weights.
- **LoRA**:
    - You also freeze the base model.
    - But you *directly* add a learnable, low-rank update inside the linear layers.

So LoRA is “closer” to full fine-tuning because it actually modifies how the network computes, but in a **structured, low-rank way** so it stays parameter-efficient.

---

7. Quick conceptual checklist (to see if you’re ready)

is not as interactive as study mode and only lists some questions the user should know

You’re in good shape for the homework if you can answer these without looking:

1. **Formulas**
    - How do we write the LoRA update for a weight matrix (W_0)?
      → (W = W_0 + BA), with (A \in \mathbb{R}^{r \times k}), (B \in \mathbb{R}^{d \times r}), (r \ll \min(d,k)).
2. **Intuition**
    - Why is this cheaper than full fine-tuning?
      → Because (r(k+d) \ll dk); we only learn a low-rank subspace of updates.
3. **Implementation**
    - Which parameters are frozen and which are trained?
    - What’s the forward pass for a LoRA-wrapped linear layer?
    - Why is (A) often initialized to zero?
4. **Usage**
    - How would you “merge” LoRA weights back into the base model for inference?

If any of these feel fuzzy, tell me which one and I can walk through a tiny numerical example or a PyTorch-style pseudo-implementation next.